

Bitcoin Fraud Detection using Graph Neural Networks

Ekaterine Tchonishvili Melanie Mariotte Shauray Sharma
Oregon State University
1500 SW Jefferson Way, Corvallis, OR 97331

tchonise@oregonstate.edu
mariottm@oregonstate.edu
sharmsha@oregonstate.edu

Abstract

Bitcoin is used by criminals for illegal activities like money laundering. Catching these criminals is difficult because there are many legal transactions. This project builds an AI system to solve this problem; it builds a full pipeline to detect illicit transactions. It starts with basic analysis to understand patterns and class imbalance. Then these results are defined as a minimum target for advanced models. Next, graph neural networks are trained. Finally, the system generates visualizations to explain the results.

1. Introduction

Cryptocurrencies such as Bitcoin have transformed modern finance by enabling decentralized transactions on a global scale. While this offers significant benefits in terms of accessibility and financial inclusion, it has simultaneously created new opportunities for financial crime, such as money laundering, scams, and illicit payments, that are difficult to detect using traditional methods.

Bitcoin transactions are inherently related. Money flows from one transaction to another, forming a directed graph where fraudulent activity rarely occurs in isolation. Criminals often operate in clusters, with illicit funds passing through chains of transactions to obscure their origins. This relational structure motivates the use of Graph Neural Networks (GNNs), which can exploit both node-level features and the topology of the transaction network simultaneously, something classical machine learning models cannot do.

We work with the Elliptic Bitcoin Dataset [1], a publicly available benchmark of over 200,000 Bitcoin transactions with ground truth labels provided by Elliptic, a blockchain analytics company. The dataset presents several challenges that are representative of real-world fraud detection: severe class imbalance (~9.76% illicit among labelled nodes), a large proportion of unlabeled transactions (~77%), and a temporal structure that creates distribution shift between training and test periods.

Our approach proceeds in two stages. We first establish strong feature-only baselines using Random Forest and Gradient Boosting, which treat each transaction independently with no knowledge of the graph structure. These baselines serve as minimum performance targets that any GNN must exceed to justify the added architectural complexity. We then train and evaluate three GNN architectures, GCN, GraphSAGE, and GAT, which leverage the full transaction graph through message passing.

The remainder of this paper is structured as follows. Section 2 reviews related work on graph-based fraud detection, GNN architecture, and class imbalance treatment. Section 3 describes our technical approach including data preprocessing, model architecture, and training procedures. Section 4 presents our experimental design and results, including an exploratory data analysis, baseline evaluation, grid search, and ablation study. Section 5 concludes with a discussion of our findings and directions for future work.

2. Related Work

Weber et al. [2] introduced the Elliptic Bitcoin Dataset and provided the first systematic evaluation of Graph Convolutional Networks for anti-money laundering in Bitcoin, demonstrating that graph structure carries meaningful signal beyond node features alone. Their work established the temporal train-test split of time steps 1-34 for training and 35-49 for testing, which we follow to ensure comparability with published benchmarks, and framed the problem as semi-supervised. Unlabelled nodes (~77%) participate in message passing despite having no ground truth label.

GNNs operate by iteratively updating node representations through message passing, where each node aggregates and transforms information from its neighbours [4]. This allows the model to encode neighbourhood structure into each node's representation, which is the core property that differentiates GNNs from the feature-only baselines we evaluate. For the practical implementation of our models, we referenced [3], which guided our use of PyTorch Geometric.

Class imbalance (only 9.76% of labelled transactions are illicit) is addressed through weighted loss functions rather than resampling strategies, following the approach of Weber et al. [2]. We evaluate both weighted cross-entropy and Focal Loss, the details of which are described in Section 3.

3. Technical Approach

3.1. Data Processing

We process the raw Elliptic Bitcoin Dataset into a single, heterogeneous PyTorch Geometric Data object, mapping transaction IDs to a contiguous integer space to form the adjacency matrix. The dataset contains 166 node-level features: the first feature represents the time step (1 to 49), while the remaining 165 consist of local transaction features and aggregated neighborhood features.

To prevent data leakage, we apply a strict temporal scaling procedure. We isolate the time step feature and fit a StandardScaler exclusively on the remaining 165 features of the training set. This scaler is then used to transform the validation and test sets before recombining them with the unscaled time step.

3.2. Baseline Modeling

We preserve the semi-supervised nature of the dataset. Nodes with "unknown" labels (-1) are retained in the graph to participate in message passing, ensuring their structural routing information is accessible to the network, but they are strictly masked out during loss computation and metric evaluation. Following the chronological nature of blockchain data, we enforce a strict temporal split: transactions from time steps 1–30 form the training set, time steps 31–34 form the validation set, and time steps 35–49 form the test set.

To establish a performance floor, we implement two non-graph, feature-only baselines: a Random Forest classifier and a Gradient Boosting classifier. These baselines are trained using a training mask covering time steps 1–34 to perfectly match the data available to the GNNs prior to the test phase. Both models utilize balanced class weighting to address the severe illicit-to-licit skew.

3.3. GNN Architectures

Next, we move on to Graph Neural Networks (GNNs). Our problem is inherently graph-structured, since transactions are interconnected with one another rather than existing independently. Because of these relationships, relying solely on baseline models that use only node-level features is not sufficient to capture the underlying patterns

in the data.

To address this, we decided to use several GNN architectures, including GraphSAGE, GAT (Graph Attention Networks), and GCN (Graph Convolutional Networks). These models are specifically designed to leverage the structure of graphs by aggregating information from neighboring nodes, allowing us to better model dependencies between transactions. As a result, they can capture both local and global structural information, which is crucial for improving performance in tasks such as detection, classification, or anomaly identification within connected data.

4. Experiments

In this section, we describe how we designed and tested our models on the dataset.

4.1. Experimental Design

We used the Elliptic dataset and split it into training, validation, and test sets based on time steps to avoid data leakage. First, we trained baseline models (Random Forest and Gradient Boosting) using only node features. Then, we trained Graph Neural Network models (GCN, GAT, and GraphSAGE) that use both node features and graph structure. To handle class imbalance, we applied class weighting and focal loss. Models were evaluated using metrics such as F1-score, precision, recall, PR-AUC, and ROC-AUC, with focus on the illicit class.

4.2. Experimental Results

4.2.1 Exploratory data analysis

We begin by analyzing the dataset to understand its structure and key patterns. The data shows a strong class imbalance: only about 9.76% of labeled transactions are illicit, while most are licit, and a large portion is unlabeled. This makes fraud detection challenging and requires models that can handle rare classes.

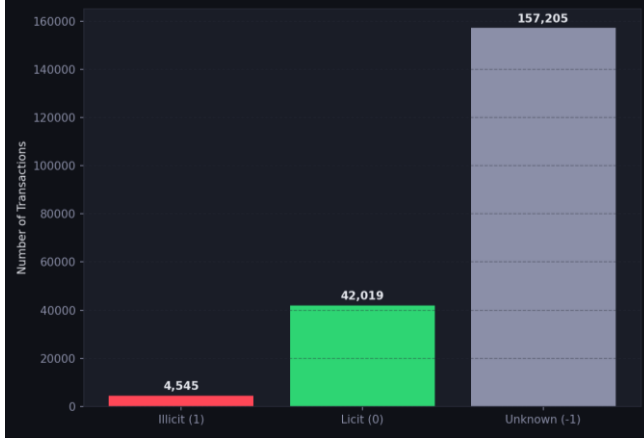


Figure 1: Node Class Distribution

We also study transaction relationships between nodes. The results show that illicit transactions tend to connect to other illicit transactions, forming clusters. This behavior, known as homophily, is important because it suggests that graph-based models can better capture these patterns than traditional models.

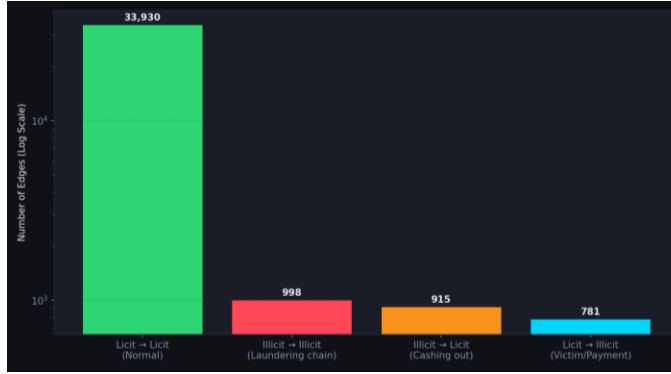


Figure 2: Transaction Counterparty Flow

Overall, the EDA highlights two key challenges: severe class imbalance and structured relationships between transactions. These findings motivate the use of Graph Neural Networks in the next experiments.

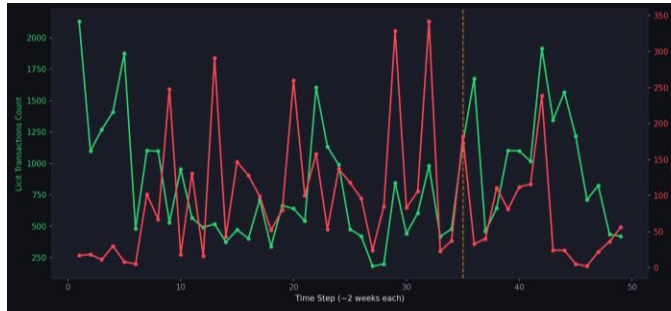


Figure 3: Temporal Transaction Volume: Licit vs. Illicit

Temporal analysis shows the evolution of transaction volumes over time. Notably, illicit activity exhibits sudden

spikes and a sharp drop around time step 43, suggesting a significant behavioral shift or intervention. In contrast, licit transactions remain relatively stable, with expected fluctuations.

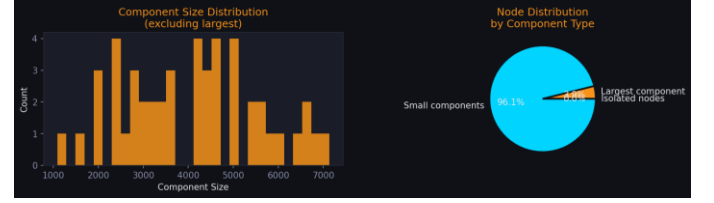


Figure 4: Graph Connected Component Analysis

Graph-level analysis demonstrates that the network is largely fragmented, with most nodes residing in small connected components (96.1%), while only a tiny fraction forms the largest connected component. Component size distribution (excluding the largest) indicates heterogeneous connectivity, which may reflect distinct transaction clusters, including potential illicit rings. These insights motivate further modeling of node classification and anomaly detection using network features.

4.2.2 Baseline

Before evaluating any GNN, we established performance targets using two feature-only baseline models trained on the 165 node features with no access to the graph structure. The results reported below are computed on the held-out test set (time steps 35-49) and serve as minimum targets that any GNN model must exceed to justify the added complexity of graph-based learning. Accuracy is deliberately excluded from all evaluation, as it is misleading under the severe class imbalance present in this dataset. A model predicting licit for every transaction would achieve over 90% accuracy while detecting zero fraud.

Model	Minority F1	PR-AUC	Precision	Recall
Random Forest	0.665	0.667	0.935	0.516
Gradient Boosting	0.613	0.708	0.658	0.574

Table 1: Baseline model performance on the test set (TS 35-49)

As shown in Table 1, the two models exhibit a clear precision-recall tradeoff. Random Forest is highly conservative. It achieves very high precision (0.935) but low recall (0.516), meaning it rarely produces false alarms but misses roughly half of all actual fraud. This behavior is confirmed by the confusion matrices in Figure 5, which show that while Random Forest correctly identifies 8,851 licit transactions (TN, 99.8%) and generates only 16 false

alarms (FP, 0.2%), it misses 216 illicit transactions entirely (FN, 48.4%). Gradient Boosting strikes a more balanced tradeoff. It catches 256 illicit transactions (TP, 57.4%) compared to Random Forest's 230, but at the cost of 133 false alarms (FP, 1.5%). In a real-world fraud detection system where missing fraud is more costly than a false alarm, Gradient Boosting's better recall makes it the more operationally useful model despite its lower Minority F1.



Figure 5: Confusion Matrices

This tradeoff is also visible in the Precision-Recall curves shown in Figure 6. Both models substantially outperform the random baseline at 0.048, which reflects the proportion of illicit nodes in the test set, confirming that the 165 node features alone carry strong discriminative signal. Both curves maintain near-perfect precision up to approximately recall=0.45, after which precision drops sharply for both models. This cliff-like behaviour is characteristic of the temporal distribution shift present in the dataset: illicit transactions most similar to the training period are caught first and easily, while the remaining fraud in the test period behaves differently enough that the model struggles to identify them without also flagging many licit transactions. Gradient Boosting consistently sits above Random Forest across the full recall range, which explains its higher PR-AUC (0.708 vs 0.667) despite a lower Minority F1.

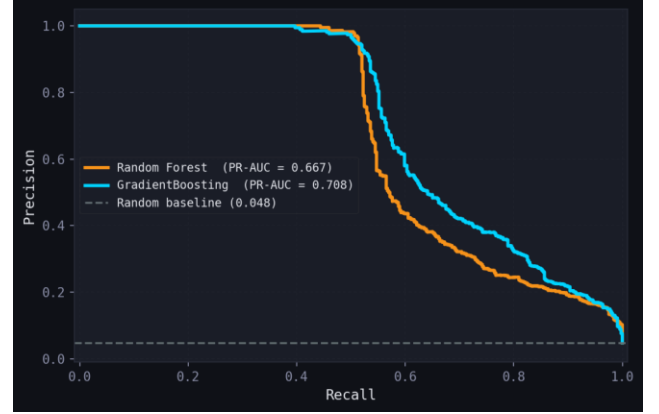


Figure 6: Precision-Recall Curves – Illicit Class

An analysis of feature importances, shown in Figure 7, reveals that local features (columns 2-94, representing per-transaction behavioral statistics) dominate the top 20 most predictive features, with only one aggregate feature (agg_39) appearing in the ranking. The time step feature does not appear at all, which is expected given that the temporal split makes it non-informative within each partition. The dominance of local features over neighbourhood-aggregated ones is a particularly relevant finding for the GNN evaluation that follows. GraphSAGE, GCN, and GAT explicitly aggregate neighbourhood features through message passing and should in theory capture richer representations than tree-based models. Whether this structural advantage translates into improved test performance is discussed in Section 4.2.2.

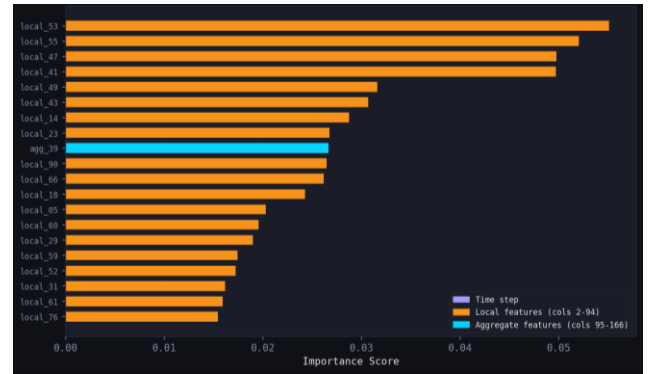


Figure 7: Feature Importance (Random Forest)

Overall, Random Forest achieves the stronger Minority F1 (0.665) while Gradient Boosting achieves the stronger PR-AUC (0.708), and both values are logged as minimum performance targets for the GNN models. It is important to emphasise that these baselines operate with no knowledge of the transaction graph. They cannot see a transaction's

neighbours, whether those neighbours are illicit, or how transactions cluster in the network.

4.2.3 GNN Results

After establishing the baseline model, we proceeded with a systematic exploration of different hyperparameters of neural networks to improve performance. For the neural network initial setup, we initially fixed the following configuration: 200 epochs, a learning rate of 0.001, and the Adam optimizer. This setup served as our reference point for further experimentation.

Building on this setup, we conducted a series of experiments by varying key training parameters. In particular, we evaluated three graph neural network architectures: Graph Convolutional Network (GCN), Graph Attention Network (GAT), and GraphSAGE. Each model was trained under different numbers of epochs—100, 200, 400, and 800—to analyze the impact of training duration on performance. Additionally, we experimented with different loss functions to observe how optimization objectives influence the results.

epoch	model_name	loss_name	test_f1	test_f1_illicit	test_precision	test_recall	test_pr_auc	test_roc_auc
0	100	gcn	weighted_ce	0.409380	0.233281	0.546991	0.746483	0.749893
1	100	gcn	Focal	0.513220	0.152601	0.521804	0.585593	0.603228
2	100	gcn	weighted_ce	0.453059	0.158722	0.522783	0.584558	0.613966
3	100	gcn	Focal	0.501639	0.223009	0.553860	0.703183	0.724566
4	100	sage	weighted_ce	0.718995	0.486184	0.682639	0.786192	0.518039
5	100	sage	Focal	0.667906	0.409081	0.633154	0.791353	0.539913
6	200	gcn	weighted_ce	0.564380	0.294656	0.582609	0.751254	0.391608
7	200	gcn	Focal	0.500729	0.228188	0.553116	0.706475	0.267482
8	200	sage	weighted_ce	0.460830	0.216808	0.555921	0.728163	0.251802
9	200	sage	Focal	0.461104	0.217453	0.556457	0.732074	0.261802
10	200	sage	weighted_ce	0.500729	0.228188	0.553116	0.706475	0.267482
11	200	sage	Focal	0.775471	0.582876	0.755503	0.799871	0.582792
12	400	gcn	weighted_ce	0.787994	0.604968	0.775280	0.802264	0.621882
13	400	gcn	Focal	0.705410	0.455796	0.681823	0.739482	0.632772
14	400	gcn	weighted_ce	0.430101	0.148946	0.518323	0.574384	0.682893
15	400	gcn	Focal	0.513824	0.248357	0.565849	0.797229	0.469734
16	400	sage	weighted_ce	0.792603	0.225113	0.553949	0.706364	0.253719
17	400	sage	Focal	0.611331	0.248357	0.799224	0.780325	0.583092
18	800	gcn	weighted_ce	0.813712	0.608797	0.827797	0.801868	0.908029
19	800	gcn	Focal	0.716502	0.471523	0.708165	0.725751	0.472470
20	800	gcn	weighted_ce	0.454785	0.158788	0.518793	0.572695	0.685213
21	800	gcn	Focal	0.504203	0.217087	0.589866	0.778175	0.381847
22	800	sage	weighted_ce	0.495092	0.222193	0.553202	0.709211	0.262882
23	800	sage	Focal	0.824386	0.659070	0.852435	0.800973	0.610078
24	800	sage	Focal	0.707413	0.621108	0.796004	0.797024	0.544093

Figure 8: Performance of GNN models (GCN, GAT, GraphSAGE) across different epochs and loss functions on the clean dataset.

For evaluation, we selected precision and recall as the primary metrics instead of accuracy. This decision was motivated by the strong class imbalance present in the dataset, where accuracy could be misleading and fail to reflect the true performance of the models, especially on minority classes.

Our experimental results indicate that GraphSAGE achieved the best performance among the tested models, particularly when tuning the number of training epochs. This suggests that its neighborhood aggregation strategy is more effective for capturing the underlying structure of the data in our specific task.

After that, we decided to compare these results with our reference model. As it turned out, the performance was worse, so we proceeded with an ablation study to better understand the impact of different components.

We experimented with several modifications. For

instance, the initial architecture included dropout, and we removed it to see whether it might improve performance. We also reduced the hidden layer size from 128 to 64, and additionally experimented with a learning rate scheduler as well as increasing the number of training epochs in these specific cases. However, despite these efforts, the reference model consistently achieved better results.



Figure 9: Ablation study of the GraphSAGE model showing the impact of individual architectural and hyperparameter changes on performance.

We hypothesize that this behavior may be caused by the data itself. Specifically, if we examine the distribution, it varies across different splits—there is a different level of imbalance in the training set compared to others, which could be affecting the model’s performance and generalization.

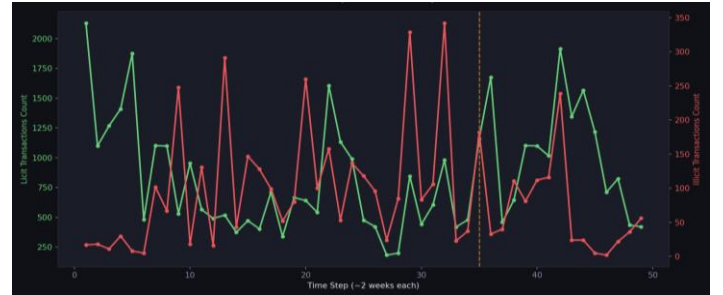


Figure 10: Temporal distribution of licit and illicit transactions over time.

After this, we experimented with different learning rates to better understand their impact on model performance. As shown in the results, extreme values led to significant degradation. Very low learning rates (e.g., 0.0001) caused the model to underfit or fail to converge within the given number of epochs, resulting in poor performance. On the other hand, very high learning rates (e.g., 0.05) led to unstable training, where the model overshoot optimal weights and performance dropped sharply.

We found that moderate learning rates (such as 0.001 and 0.005) provided the best balance between convergence speed and stability, leading to significantly better results across evaluation metrics. This experiment highlights the sensitivity of graph neural networks to learning rate selection and reinforces the importance of careful hyperparameter tuning.



Figure 11: Heatmap performance comparison of GraphSAGE models using Weighted Cross-Entropy and Focal Loss across various learning rates.

After this, we experimented with different optimizers to evaluate their impact on model performance. Specifically, we compared stochastic gradient descent (SGD), Adam, and AdamW under the same training setup.

The results show that SGD-based configurations consistently underperformed across all evaluation metrics, particularly in terms of F1 score and PR AUC, indicating slower convergence and less effective optimization for this task. In contrast, Adam-based models achieved significantly better and more stable results, confirming its suitability for training graph neural networks in our setting.

Between Adam and AdamW, the differences were relatively small, although AdamW provided slightly more consistent performance in some cases due to its improved regularization through weight decay. Overall, the best-performing configurations were obtained using the Adam optimizer combined with both weighted cross-entropy and focal loss, demonstrating strong performance across precision, recall, and F1 metrics.

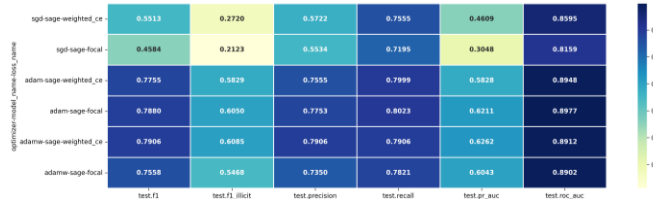


Figure 12: Comparison of optimizer performance (SGD, Adam, AdamW) across Weighted Cross-Entropy and Focal Loss.

In the original dataset we have regular and aggregated features. We tried to evaluate our data different ways – see our model results using both features, for only regular ones and for only aggregated ones. As expected, both features should be included for best results.

Feature Set	Test Illicit F1	Test Precision	Test Recall	Test PR-AUC
Both	0.592	0.557	0.632	0.605
Regular Only	0.523	0.496	0.552	0.511
Aggregated Only	0.496	0.415	0.616	0.490

Table 2: Feature ablation comparison results for GraphSAGE using weighted cross-entropy, Adam, a learning rate of 0.001, the strict temporal split, and excluded unknown nodes.

Feature set table compares performance when training on regular features only, aggregated features only, and both feature groups combined. While regular features describe the transaction itself, aggregated features summarize the information from neighboring transactions.

From these experiments, the results indicate that our model performs the best by using both regular and aggregated features together.

Split Mode	Test Illicit F1	Test Precision	Test Recall	Test PR-AUC
Strict	0.592	0.557	0.632	0.605
70/30	0.630	0.629	0.631	0.623
75/25	0.579	0.663	0.514	0.569
80/20	0.634	0.799	0.525	0.636

Table 3: Temporal split ablation results for GraphSAGE using weighted cross-entropy, Adam, a learning rate of 0.001, both feature groups, and excluded unknown nodes.

Temporal split ablation table compares the original strict temporal split against alternative timestep partitions which are 70/30, 75/25, and 80/20. The alternative timestep modes allocate different proportions (than the strict timestep mode) of the timeline to train/validate vs test while still preserving the temporal order. The team decided to do a temporal split because the elliptic dataset is organized by timestep, and also because it is a little more realistic than a random split. It forces the model to learn from earlier transactions and generalize to later ones. The strict mode, which is the original temporal split, consists of training on timesteps 1 - 30, validating on timesteps 31 - 34, and testing on timesteps 35 - 49. Performance improved under alternative temporal splits, with the 80/20 split giving the highest illicit F1 and PR-AUC.

Futhermore, the team decided to investigate unknowns and compare results with unknowns included and excluded from the model.

Unknown Handling	Test Illicit F1	Test Precision	Test Recall	Test PR- AUC
Unknowns Excluded	0.533	0.459	0.634	0.525
Unknowns Included	0.319	0.226	0.541	0.230

Table 4: Unknown handling results for GraphSAGE using weighted cross-entropy, Adam, a learning rate of 0.001, both feature groups, and the strict temporal split.

The unknown handling test evaluates whether if the unknown transactions should be incorporated into the model or if they should be excluded from the model. In the baseline GNN configuration, the unknown nodes remain in the graph so that they contribute structural information through message passing; however, they are excluded from the loss because their label is not part of the binary target task. In our alternative setting, the unknown nodes are treated as third class and included in training, validation, and testing. The team considered this experiment to be important because there were many unknowns in the dataset and because unknowns may influence our model’s learning. The results indicate that including the unknowns worsened our illicit F1, precision, recall, and PR-AUC than when we had excluded them from the model. It is most likely that since we forced the model to learn a third category, it made the classification problem harder which could have also worsen our results for this experiment.

5. Conclusion and Future Work

Our experiments demonstrate that GraphSAGE is the most effective architecture among the evaluated graph neural networks, consistently outperforming alternatives across key metrics.

The optimal configuration combines GraphSAGE with the Adam optimizer and a learning rate of 0.005, achieving the best balance between convergence speed and overall performance.

For future work, several directions can further improve the system. First, adopting a better temporal data split could reduce distribution shifts and improve generalization. Additionally, exploring Temporal Graph Networks may allow the model to better capture time-dependent relationships in the data. Finally, incorporating Transformers could provide more powerful representation learning capabilities, especially for complex interaction patterns.

References

- [1] Elliptic, www.elliptic.co.

- [2] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, C. E. Leiserson, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics", KDD '19 Workshop on Anomaly Detection in Finance, August 2019, Anchorage, AK, USA.
- [3] <https://www.kaggle.com/code/iogbonna/introduction-to-graph-neural-network-with-pytorch>
- [4] B.Sanchez-Lengeling, E. Reif, A. Pearce, A. Wiltchko, "A Gentle Introduction to Graph Neural Networks", Distill, 2021. <https://distill.pub/2021/gnn-intro/>